

MAY 2026

Vision-Based Precision Landing System for Autonomous UAVs Using AprilTag Fusion

A PROJECT REPORT

Submitted by

BL.EN.P2RAU24001

AKASH DINESH

*in partial fulfillment for the award of the degree
of*

MASTER OF TECHNOLOGY

IN

“Robotics and Automation”



AMRITA SCHOOL OF ENGINEERING, BANGALORE

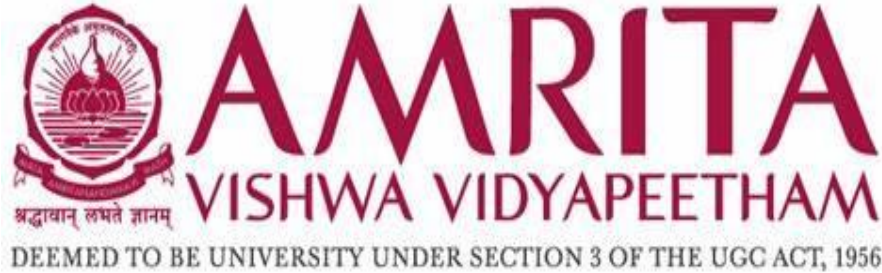
AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

MAY-2026

MAY 2026

AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ENGINEERING, BENGALURU, 560035



BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**Vision-Based Precision Landing System for Autonomous UAVs Using AprilTag Fusion**” submitted by,

BL.EN.P2RAU24001

AKASH DINESH

in partial fulfillment of the requirements for the award of the **Master of Technology** in “**ROBOTICS AND AUTOMATION**” is a bonafide record of the work carried out under my(our) guidance and supervision at Amrita School of Engineering, Bengaluru.

Dr. Sreeja Kochuvila
Guide
Assistant Professor (Sl. Gd.)
Department of ECE

Dr. T.K. Ramesh
Chairperson
Professor
Department of ECE

Dr. Manoj Kumar Panda
Associate Professor
Department of ECE
EXAMINER 1

Dr. Rajeevlochana G. Chittawadigi
Assistant Professor (Sl. Gd.)
Department of ECE
EXAMINER 2

ACKNOWLEDGMENT

I take this opportunity to express my gratitude to everyone who contributed to the completion of this project. First and foremost, I would like to express my sincere gratitude to our beloved Amma, Mata Amritanandamayi Devi for her blessings.

I would like to express my sincere gratitude to our Director Dr. Manoj P. for the successful completion of the project. I would like to extend my sincere thanks to our principal Dr. Sriram Devanathan for inspiring me. I am highly indebted to Dr. T. K. Ramesh Chairman, Department of Electronics and Communications Engineering for motivating me towards the successful completion of the project.

My heartfelt thanks to Dr. Sreeja Kochuvila (Assistant Professor (Sl. Gd.)) Department of Electronics and Communication Engineering, Dr. Manoj Kumar Panda (Associate Professor) Department of Electronics and Communication Engineering, Dr. Rajeevlochana G. Chittawadigi (Assistant Professor (Sl. Gd.)) who have been very patient and helpful at all times and pointing me in the right direction.

A special thanks to all staff members of the Department of Electronics and Communication Engineering for their support and guidance. I shall remember fondly the wholehearted support of my family throughout the progress of the project.

Akash Dinesh

ABSTRACT

Unmanned Aerial Vehicles (UAVs) depend heavily on the Global Navigation Satellite System (GNSS) for autonomous navigation and mission execution. However, GNSS positioning near ground level is limited to one-to-three metres of horizontal accuracy due to multipath effects, signal drift, and atmospheric error, which is inadequate when a landing pad is only a fraction of a metre wide. This project presents a complete onboard precision-landing system that closes this gap using fiducial-marker computer vision fused with the ArduPilot precision-landing framework.

The work proceeds in two phases. The first phase implemented the conventional approach found throughout the ArduPilot ecosystem: detection of a single 30 cm ArUco (4×4) marker using OpenCV on a Raspberry Pi 4B, with relative pose transmitted to a Matek H743-SLIM V3 flight controller over MAVLink. This baseline validated the end-to-end hardware chain but exposed two fundamental limitations — a single fixed marker size cannot remain both detectable at altitude and within the field of view during terminal descent, and the uncalibrated pipeline produced altitude-dependent scale errors.

The second phase, which constitutes the primary contribution, replaces the single ArUco marker with a multi-scale AprilTag (tag36h11) layout: one 24 cm central tag surrounded by four 8 cm corner tags on a 60 cm board, ensuring continuous detection across the full descent range. A weighted multi-tag fusion estimator combines per-tag pose estimates into a single landing-centre position, weighted by detected pixel area, decoder margin, and image-edge safety. The pipeline is supported by a calibrated camera model (chessboard calibration, 0.255 px RMS reprojection error), per-frame undistortion, locked sensor mode, and a frame-to-frame jump filter that suppresses vibration-induced detection noise.

The system was validated through powered hover flights from 1.7 m to 8.6 m altitude. Internal multi-tag pose agreement was measured at 5.0 cm at 1.74 m with all five tags visible, and 1.1 cm at 3.63 m with two tags visible, demonstrating strong geometric self-consistency on real hardware. The central tag was reliably detected up to 8.56 m. The system additionally demonstrated

genuine inclined-surface tilt estimation, distinguished rigorously from single-tag pose artifacts by the behaviour of the disagreement metric under fusion. The report documents a set of non-obvious hardware-integration pitfalls — coordinate-frame sign conventions, square-marker pose ambiguity, MAVLink timestamp formatting, camera miscalibration, sensor-mode locking, and vibration noise — that are common in deployment but underdocumented in the literature. The system is well suited to autonomous docking, last-mile delivery, inspection, and recharging-station applications.

TABLE OF CONTENTS

CHAPTER-1: Introduction	1-6
1.1 Introduction	1
1.2 Background and Motivation	1
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope and Contributions	4
1.6 Organization of the Report	6
1.7 Summary	6
 CHAPTER-2: Literature Review	 7-12
2.1 Introduction	7
2.2 Vision-Based Navigation and UAV Pose Estimation	7
2.3 Fiducial Marker Families: ArUco vs AprilTag	8
2.4 Nested and Multi-Scale Marker Layouts	8
2.5 Fiducial-Based Landing and Multi-Sensor Systems	9
2.6 Companion Computer and ArduPilot Integration	9
2.7 Hybrid GNSS-Vision and Sensor-Fusion Approaches	10
2.8 Lightweight Onboard Computer Vision for Small UAVs	11
2.9 Summary	11
 CHAPTER-3: System Design and Hardware Architecture	 13-22
3.1 Introduction	13
3.2 Overview of the Proposed System	13
3.3 Airframe and Propulsion	15
3.4 Flight Controller and Companion Computer	18
3.5 Camera and Sensor Subsystem	19
3.6 Communication and Power Architecture	19
3.7 Marker Design	20
3.8 Summary	22

CHAPTER-4: Software Architecture and Methodology 23-31

4.1	Introduction	23
4.2	Phase 1: ArUco Baseline Pipeline	23
4.3	Phase 2: Multi-Tag AprilTag Pipeline	24
4.4	Camera Calibration and Undistortion	25
4.5	Per-Tag Pose Estimation	26
4.6	Coordinate-Frame Transformation	27
4.7	Multi-Tag Fusion Estimator	28
4.8	Velocity Estimation and Lag Compensation	29
4.9	MAVLink Integration	29
4.10	Vibration Jump Filter and Robustness	30
4.11	Deployment, Logging and Recording	30
4.12	Summary	31

CHAPTER-5: Experimental Setup and Testing Methodology 32-36

5.1	Introduction	32
5.2	Bench Testing	32
5.3	Powered Hover Testing	33
5.4	ArduPilot Parameter Configuration	33
5.5	Date Collection and Log Correlation	35
5.6	Summary	35

CHAPTER-6: Results and Discussion 37-46

6.1	Introduction	37
6.2	Hardware Implementation Results	37
6.3	ArUco Baseline Detection Results	37
6.4	AprilTag Multi-Tag Detection and Fusion Results	38
6.5	Detection Continuity Across Altitude	40
6.6	Single-Tag Tilt Instability and Inclined-Platform Detection	41
6.7	Documented Integration Pitfalls	44
6.8	Discussion	45

6.9	Summary	46
CHAPTER-7: Conclusion and Future Work		47-49
7.1	Conclusion	47
7.2	Future Work	47

LIST OF FIGURES

Figure No.	Figures	Page No.
3.1	System architecture and data flow diagram	8
3.2	Top-down view of the assembled 5-inch iFlight XL5 quadrotor	9
3.3	Side profile of the airframe showing vertical hardware stack	31
3.4	Printed multi-scale AprilTag landing board (5 tags, 60 cm board)	43
4.1	Per-frame software pipeline block diagram	44
6.1	Low-altitude detection at 1.74 m with all five tags visible	45
6.2	Mid-altitude two-tag fusion at 3.63 m	46
6.3	Maximum-altitude single-tag detection at 8.56 m	47
6.4	Single-tag tilt instability at 6.28 m over a level board	48
6.5	Inclined-platform detection at 1.68 m with full five-tag fusion	49

ACRONYMS

UAV	Unmanned Aerial Vehicle
FC	Flight Controller
GPS	Global Positioning System
GNSS	Global Navigation Satellite System
UART	Universal Asynchronous Receiver– Transmitter
ESC	Electronic Speed Controller
ELRS	ExpressLRS (Long-Range RC Protocol)
MAVLink	Micro Air Vehicle Link
GCS	Ground Control Station
VIO	Visual-Inertial Odometry
FPS	Frames Per Second
IMU	Inertial Measurement Unit
Li-Ion	Lithium-Ion Battery
SSH	Secure Shell
ArUco	Augmented Reality University of Córdoba (marker library)
UAVCAN	Unmanned Aerial Vehicle Controller Area Network
PLND	Precision Landing Parameter Set (ArduPilot)

CHAPTER 1

INTRODUCTION

1.1 Introduction

This chapter introduces the precision-landing problem, establishes why GNSS alone is insufficient for the terminal phase of an autonomous mission, and sets out the motivation, problem statement, objectives, scope, and contributions of the project. It closes by describing how the remainder of the report is organised.

1.2 Background and Motivation

UAVs have made quick strides from specialist systems to widespread usage in such fields as mapping, surveillance, infrastructure assessment, agriculture, and last-mile autonomous deliveries. In these developments, the need has evolved from merely sustaining flight and navigation to completing a full mission with little human involvement. With respect to autonomous operations, landing becomes particularly crucial since it marks the point of greatest danger for an aircraft, where it confirms its estimate with the actual surroundings, thus determining success or failure in a particular task.

During the cruise and approach phases of the flight, GNSS offers adequate guidance. Consumer-grade GNSS can determine location with an accuracy of one to three metres, enough for travelling from point to point several metres apart. The problem starts exactly when the flight ends. In the close vicinity of the ground, the GNSS signal loses additional accuracy due to multipath effect caused by nearby buildings; while one-to-three-metre error is insignificant on cruise altitude, such accuracy becomes critical as the target location – landing place – may be just twenty to sixty centimetres wide. A GNSS-only landing onto such a target is, in effect, a guess.

Closing this final-approach accuracy gap requires an additional sensing modality referenced to the landing target itself rather than to a global frame. Among the available

options — visual-inertial odometry, LiDAR, optical flow, RTK GNSS — vision-based fiducial detection has emerged as the dominant solution for small UAV platforms, because it is inexpensive, lightweight, computationally tractable on embedded hardware, and directly measures the quantity of interest: the offset between the aircraft and the pad. A downward-facing camera detects a printed marker of known size, recovers the relative pose, and reports the offset to the autopilot, which corrects its descent to drive that offset to zero.

This project develops, deploys, and validates such a system end-to-end on real hardware, and in doing so traverses the gap between the textbook description of fiducial landing and the considerably messier reality of making it work on a vibrating, power-constrained, real-time airframe.

1.3 Problem Statement

The core problem addressed in this work is the design and hardware implementation of a low-cost, fully onboard precision-landing system for a small multirotor UAV that achieves sub-decimetre relative-position accuracy against a fixed landing target, without dependence on RTK GNSS, LiDAR, or GPU-class computing.

This top-level problem decomposes into several sub-problems that the project addresses in sequence. The first is the range–field-of-view conflict inherent to single-marker systems. A marker large enough to be decoded from several metres of altitude is too large to remain wholly within the camera frame during the final metre of descent; conversely, a marker small enough to stay in frame near touchdown falls below the sensor’s resolution limit at altitude. A practical system must maintain continuous detection across the entire descent.

The second difficulty in the use of this system for landing UAVs relates to the accuracy of the metric reconstruction of the recovered pose. The reconstruction of the position relative to the marker is based on the intrinsic parameters of the camera and, therefore, on the correctness of its calibration. Unspecified or inaccurate camera parameters will yield an apparent estimate of the drone's position that is stable in the

camera frame but not accurate with respect to metric coordinates in the real world. Such inaccuracies are prone to grow with the altitude, which will cause the controller to compensate using incorrect values.

The third issue is the robustness of the solution on a real aircraft. Firstly, during powered flights, the movement of engines and rotors will cause vibration noise, adding up to image blurring and jitter. Moreover, electrical noise may interfere with data acquisition, while onboard calculations delay may negatively affect the stability of the control loop. Finally, it is vital to correctly align the coordinate systems used by the visual system, autopilot, and cameras themselves, since any difference in axes convention may lead to erroneous compensations of movements. The fourth, explored as an extension, is operation over non-level landing surfaces, where the landing target is inclined relative to the horizontal and the system must recover not only lateral offset but the orientation of the surface.

1.4 Objectives

The specific objectives of the project are:

1. To design and physically build a 5-inch multirotor UAV integrating a flight controller, companion computer, downward-facing camera, GNSS, and long-range radio, with vibration-isolated custom mounts.
2. To establish a reliable MAVLink communication link between the companion computer and the flight controller, and to verify the transmission of relative-position data via the LANDING_TARGET message at the camera frame rate.
3. To implement a baseline single-marker ArUco detection-and-pose pipeline on the companion computer using OpenCV, validating the complete hardware-and-software chain.
4. To design a multi-scale, multi-tag AprilTag landing target that maintains continuous detection across the full descent altitude range.
5. To implement a weighted multi-tag fusion estimator that combines per-tag pose estimates into a single robust landing-centre position and surface orientation.

6. To calibrate the camera and integrate a full undistortion and sensor-mode-locking pipeline that preserves metric pose accuracy across altitude.
7. To validate the system through bench and powered-hover flight tests, characterising detection continuity, multi-tag agreement, and behaviour over both level and inclined landing surfaces.
8. To document the hardware-integration challenges encountered and the diagnostic methodology used to resolve them, as a practical reference for similar deployments.

1.5 Scope and Contributions

The problem space addressed by this project includes the entire pipeline of perception to autopilot precision-landing solution and hardware realization thereof onto a resource-restricted unmanned aerial vehicle (UAV). The solution is a combination of GNSS navigation used at greater ranges and coarser approaches, along with a vision-based precision-landing algorithm employed for final fine adjustment towards landing. Specifically, when a UAV lands, it first approaches the goal position based on GNSS coordinates; subsequently, the vision module takes over to provide finer corrections by recognizing AprilTag targets positioned on the ground plane. Real-time calculation of the estimated relative position and orientation of the vehicle is performed via the vision pipeline and then sent to the autopilot using the precision-landing interface protocol to control autonomous landing.

Configuration and testing of the autonomous landing controller involve the precise-landing capabilities provided by the ArduPilot flight software stack. Tuning of the necessary parameters associated with precise landing ensures that the drone can descend steadily, correct its position smoothly, and respond to touchdowns consistently while allowing for a manual override function. The autonomous controller has been shown to provide reliable autonomous navigation and landing on flat and inclined surfaces through experimental validation. Despite the success in estimating the slope

of the landing surface and descending under control, the next development phase is autonomous touchdown.

The key innovations made in this paper are listed below:

1. The development of an AprilTag marker design consisting of multiple scales and multiple tags in combination with a quality-weighted fusion estimator for the fusion of pose estimates of multiple tags that are seen in view. The fusion estimator constantly assesses consistency among the tags, and it uses it as a metric for detecting unreliable detections, thereby increasing robustness under partial occlusion, fast motions, and different altitudes.
2. Hardware implementation in an embedded system with the size of a 5-inch quadrotor drone, which detects and fuses information about the tags in real-time at the speed of a camera feed frame. This setup runs purely on board, without any need for wireless communication with an external computer.
3. Inclination platform detection and tilt angle estimation of the landing surface via multi-tag pose fusion analysis. It is shown through experimental testing that the detected tilt angles of the surface persist and remain geometrically consistent despite performing full multi-tag pose fusion with sub-centimeter positional consistency among all the tags. The above mentioned observation separates real landing surface tilts from the geometric artifacts created due to the use of single tag pose estimation methods.
4. Documentation of practical problems encountered when integrating hardware in practice, along with solutions used to tackle the encountered problems. These include coordinate frame sign convention issues, ambiguity with square markers' poses, timestamp format differences between MAVLink, calibration error of cameras, sensors mode locking behavior, and vibration induced noise.

1.6 Organization of the Report

Chapter 2 discusses the relevant literature across vision-based navigation, fiducial marker families, nested marker layouts, autopilot integration, and lightweight onboard vision. Chapter 3 details the system design and hardware architecture. Chapter 4 presents the software architecture and methodology, covering both the ArUco baseline and the AprilTag fusion pipeline. Chapter 5 describes the experimental setup and testing methodology. Chapter 6 presents and discusses the results. Chapter 7 concludes and identifies future work.

1.7 Summary

In summary, this chapter has established the need for vision-based precision landing to overcome the terminal-phase limitations of GNSS, framed the four sub-problems the project addresses — detection continuity, metric pose accuracy, real-airframe robustness, and operation over non-level surfaces — and defined the objectives, scope, and contributions that the remaining chapters develop.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter surveys the body of work on which the project builds. It reviews vision-based UAV navigation and pose estimation, compares the ArUco and AprilTag fiducial families, examines nested and multi-scale marker layouts, and considers fiducial-based landing systems, companion-computer integration with ArduPilot, hybrid GNSS-vision strategies, and the feasibility of onboard vision on embedded hardware, concluding with a consolidated summary of the survey.

2.2 Vision-Based Navigation and UAV Pose Estimation

Yang et al. proposed a vision-based autonomous navigation framework for quadrotor UAVs operating in unknown and constrained outdoor environments, integrating visual-inertial odometry with path planning so that stable trajectories are maintained even under degraded GNSS conditions. In their work, Sepahvand et al. have emphasized an even greater principle that underpins the current project: visual perception is able to assist or take over the function of GNSS in cases when an exact estimation of the ground-referenced position is needed. This is particularly important in the case of landing since precise estimation of the relative position is critical for a stable and safe landing process. While the use of GNSS is enough for navigation, its position uncertainties make it unsuitable for precision landing procedures, especially when the platform is tilted or small.

In addition, Sepahvand et al. suggested applying a visual-servoing solution based on CNN extracted image features for real-time attitude and orientation control of the UAV. The authors managed to attain very high levels of accuracy and robustness in feature detection; however, computational costs of the continuous CNN inference are rather significant. For real-time processing, special GPU-grade hardware is usually required, which makes the approach inapplicable for embedded lightweight systems

like Raspberry Pi utilized in the current research. Thus, the current paper introduces another motivation for using classical fiducial marker detection algorithms.

Scaramuzza and Fraundorfer's foundational treatment of visual odometry, and Kendoul's survey of guidance, navigation, and control for unmanned rotorcraft, together frame the field within which fiducial-based landing sits: a special, highly structured case of visual pose estimation in which the environment is deliberately instrumented with a marker of known geometry, trading generality for accuracy and reliability.

2.3 Fiducial Marker Families: ArUco vs AprilTag

ArUco markers have been proposed by Garrido-Jurado et al., and they are an automated method for generating and detecting fiducial markers with customizable dictionaries as well as error-correction capabilities. ArUco has become very popular in many UAV landing applications and is still used extensively in ArduPilot documentation because of its compatibility with OpenCV's aruco module and efficiency. This project utilizes ArUco in the initial phase of development due to its reliability and efficiency as compared to other approaches.

AprilTag markers were introduced by Olson and improved further in terms of decoding margins and resilience against motion blur, as well as partial occlusions by Wang & Olson. The tag36h11 family in particular has become the predominant choice for aerial precision landing. Kalaitzakis et al. conducted a systematic experimental comparison of ARTag, AprilTag, ArUco, and STag markers for pose estimation, consistently ranking AprilTag favourably for outdoor and aerial use in terms of detection robustness and pose stability. This comparative evidence underpins the project's second-phase migration from ArUco to AprilTag.

2.4 Nested and Multi-Scale Marker Layouts

Single-marker systems suffer the fundamental range limitation described in Section 1.2. Wang et al. presented an autonomous tracking-and-landing method for

quadrotor UAVs using nested ArUco markers, placing markers of different sizes in concentric patterns so that marker visibility is maintained continuously as the aircraft descends. Krogus, Haggemiller, and Olson generalised this idea with flexible layouts for fiducial tags, formalising the use of multiple tags in a known geometric arrangement to extend the operational range and improve robustness. These works provide the conceptual foundation for this project's multi-scale layout — one large central tag for altitude, four small corner tags for terminal descent — and for treating the multiple per-tag estimates as a fusion opportunity rather than merely a redundancy.

2.5 Fiducial-Based Landing and Multi-Sensor Systems

Patrino et al. developed a vision-based approach for UAV landing using a fiducial marker and demonstrated reliable pose recovery for autonomous descent. Marut, Wójtowicz, and Falkowski specifically studied ArUco-marker pose estimation within a UAV landing-aid system, confirming the practicality of the approach in the landing context and reporting the lateral and yaw corrections achievable before touchdown.

Springer et al. extended fiducial landing into challenging lighting by combining visual and infrared fiducial markers with a multi-payload camera, fusing RGB and IR data to maintain marker visibility across day and night operations; in later work, Springer and Kyas added a gimbal-mounted camera for active target tracking. While effective, these multi-sensor and actuated-camera designs add hardware complexity and payload that are unattractive on a compact 5-inch airframe. The present project deliberately constrains itself to a single fixed RGB camera, emphasising low cost and minimal hardware dependency, and recovers robustness instead through marker design and software fusion.

2.6 Companion Computer and ArduPilot Integration

The ArduPilot documentation provides extensive guidance for integrating companion computers, implementing guided and precision landing, and fusing sensor data, supporting MAVLink communication, optical-flow inputs, and rangefinder-assisted descent. In the canonical architecture, the flight controller retains

responsibility for low-level stabilisation and safety while the companion computer performs high-level perception and decision-making — a separation of concerns that simplifies the design of complex behaviours such as precision landing and is adopted directly in this project.

Baca et al. investigated embedded computer vision for UAV autonomous landing using MAVLink and companion computers, demonstrating a hybrid strategy in which GNSS handles coarse waypoint navigation and an external computer injects refined corrective setpoints during landing. Their findings validate the architecture used here, in which a Raspberry Pi 4B communicates relative-position estimates to a Matek H743-SLIM V3 over UART to guide the final descent.

2.7 Hybrid GNSS–Vision and Sensor-Fusion Approaches

A consistent theme across the literature is that hybrid systems combining GNSS and visual feedback outperform either modality alone for landing. Lange, Sünderhauf, and Protzel presented an early vision-based onboard approach for landing and position control of a multirotor in GNSS-denied environments, establishing that vision alone can close the terminal-descent loop. Luo, Qin, and Shen demonstrated monocular visual-inertial state estimation for autonomous drone landing, fusing camera and inertial data for smooth transitions between global and local reference frames and stable vertical and lateral control during final descent.

For landing on moving targets — a natural extension of static-pad landing — Borowczyk et al. demonstrated autonomous landing of a quadcopter on a high-speed ground vehicle, and Falanga et al. achieved vision-based autonomous quadrotor landing on a moving platform, both relying on prediction and lag compensation to track a target whose position evolves during descent. These works inform the velocity-estimation and lag-compensation components implemented in this project’s pipeline, which are present in the software and provide the foundation for the dynamic-platform extension identified as future work.

2.8 Lightweight Onboard Computer Vision for Small UAVs

The feasibility of running real-time fiducial detection on embedded hardware is well established. Studies of OpenCV-based marker detection on Raspberry-Pi-class processors confirm that classical computer-vision techniques achieve frame rates sufficient for responsive descent control without dedicated GPUs, with solvePnP-based pose estimation providing the deterministic latency that closed-loop landing requires. Corke’s treatment of fiducial markers and pose estimation for robotics provides the underlying geometric framework. Together, these works confirm that edge-based computer vision is practical for small 5-inch UAV platforms and support the project’s decision to perform all perception onboard.

2.9 Summary

- Vision-based perception is essential for improving UAV localisation and stability when GNSS degrades, and the landing phase is the canonical application where a reliable ground reference is both available and necessary.
- Deep-learning visual servoing achieves high precision but demands heavy computation, validating the choice of classical fiducial detection for a lightweight, real-time embedded system.
- AprilTag (tag36h11) markers are experimentally shown to deliver higher decoder margins and greater pose robustness than ArUco for aerial use, motivating the project’s migration from an ArUco baseline to an AprilTag system.
- Nested and multi-scale marker layouts resolve the fundamental range–field-of-view conflict of single-marker systems and naturally provide multiple pose estimates suitable for fusion.
- Multi-sensor and actuated-camera landing systems improve robustness at the cost of hardware complexity; a single fixed RGB camera with software fusion is preferable on a compact airframe.

- The companion-computer/flight-controller separation over MAVLink is the established and reliable architecture for adding precision landing to an ArduPilot vehicle.
- Hybrid GNSS-coarse / vision-fine strategies significantly reduce landing error and are the consensus approach.
- Prediction and lag compensation are the established tools for extending static-pad landing to moving platforms.

CHAPTER 3

SYSTEM DESIGN AND HARDWARE ARCHITECTURE

3.1 Introduction

The chapter describes the actual implementation of the system. The chapter starts with the introduction of the two-subsystem configuration, followed by the description of airframe and propulsion, the flight controller and the companion computer, the camera and sensors subsystem, communication and power, as well as the multi-scale AprilTag landing target.

3.2 Overview of the Proposed System

The design of the proposed landing precision system relies on a downward-looking camera detecting a scale invariant fiducial marker, estimating the relative pose of the landing target, and relaying this information back to the flight controller via MAVLink messages.

From the mission perspective, the design adopts the GNSS-vision approach as described in the relevant literature: the aircraft flies to the vicinity of the landing site using GNSS waypoints, and further aligns itself at close range using only vision. The architecture of the design implies that there are two physically distinct systems that functionally interact with each other in tight cooperation. The companion computer, which uses a Raspberry Pi 4B board, takes care of all perception-related tasks: capturing the image frame, detecting fiducial markers, computing per-tag pose estimates, fusing multiple tags, performing coordinate transformations and packing data into MAVLink messages.

The flight controller (Matek H743-SLIM V3) remains responsible for vehicle stabilisation, modes and safety, as well as closed-loop controlling of the descent trajectory based on position estimates provided by the companion computer.

This separation of concerns is deliberate and follows the canonical ArduPilot companion-computer architecture: the safety-critical real-time control runs on dedicated, certified firmware, while the computationally heavier and more experimental perception logic runs on a general-purpose Linux computer where it can be developed and iterated rapidly.

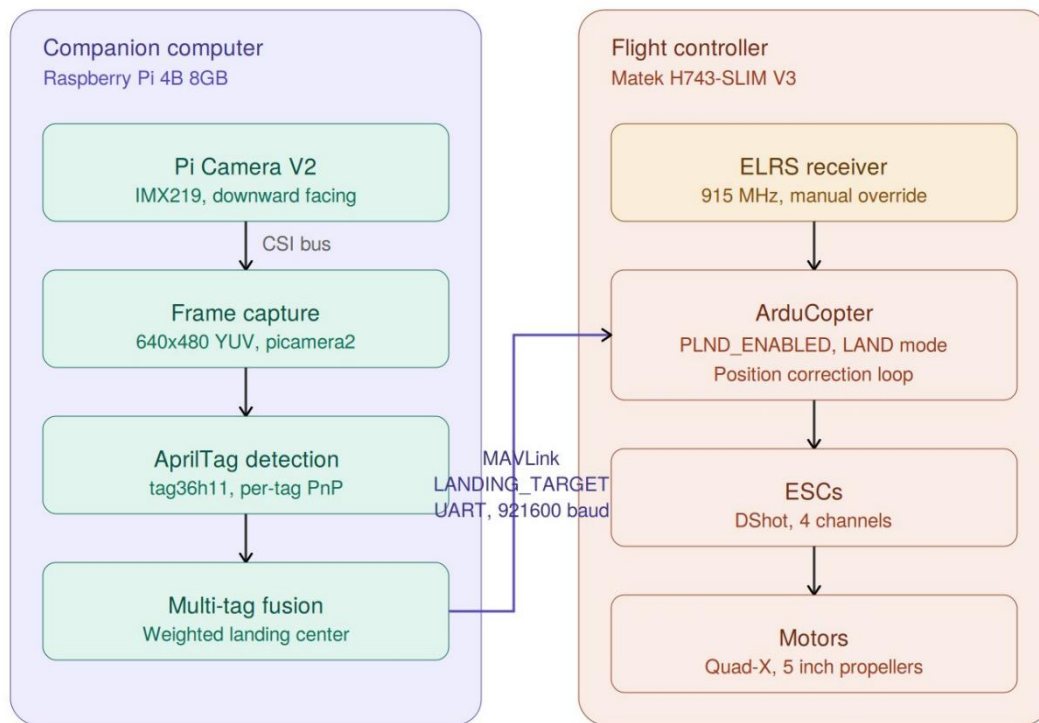


Fig. 3.1 System architecture and data flow diagram

The architecture, shown in fig.3.1, makes the data flow explicit. The Pi Camera V2 supplies frames to the Raspberry Pi over the CSI bus. The detection-and-fusion pipeline runs in Python and produces LANDING_TARGET messages, which are transmitted over a UART link to the flight controller. These correctional signals will be integrated into the control process by the flight controller during the LAND phase using the four channels for the DShot ESC/motors. Manual control is also possible at all times via the 915 MHz ExpressLRS radio communication system.

3.3 Airframe and Propulsion

The airframe is a specially designed 5-inch quadrotor with the iFlight XL5 airframe layout in the Quad-X variant. Choosing the 5-inch class as the minimum size was driven by the requirement to be able to mount the entire perception sensor suite, consisting of the companion computer, camera, GPS, and radio link, while still maintaining enough power margin and flight time for all of the hovering and descending tests needed for the project. The airframe body consists mainly of carbon fiber sheets.

The propulsion system consists of iFlight Xing 2207 2450 KV brushless motors that spin 5-inch tri-blade propellers, via a Skystars KM50 50 A quad ESC, running the DShot communication protocol. The electric power supply comprises a 4S Li-ion 4200 mAh battery pack, selected over a traditional Li-Po, due to its higher energy density and increased hovering capacity, which is beneficial for the lengthy characterisation/detection missions carried out during testing.

All structural elements except those used to mount hardware, including the GNSS aerial, camera mount, Raspberry Pi case, and four legs of the landing gear, have been specially fabricated and 3D-printed. Beyond simple fit, these parts serve a vibration-management function: the camera bracket in particular rigidly fixes the sensor to the frame in a known orientation, which is essential because the entire coordinate-transformation chain depends on the camera being mounted at a fixed, known attitude relative to the airframe. The complete hardware inventory is summarised in Table 3.1.

Table 3.1 Hardware components of the UAV platform

Airframe	iFlight XL5 5-inch carbon-fibre, Quad-X
Flight Controller	Mateksys H743-SLIM V3 (ArduCopter / ArduPilot)
ESC	Skystars KM50 50 A four-in-one (DSHOT)
Motors	iFlight Xing 2207 2450 KV
Battery	4S Li-ion 4200 mAh
Companion Computer	Raspberry Pi 4B 8GB
Camera	Pi Camera V2 (Sony IMX219), downward-facing
GNSS	GEPRC M10-DI
Radio	915 MHz ExpressLRS (RadioMaster TX, onboard RX)
Power Regulation	Hobbywing UBEC (5 V rail for Pi)
3D-printed parts	GPS mast, Pi holder, camera bracket, landing gear ×4
Companion link	MAVLink2 over UART (SERIAL6, 921600 baud)



Fig. 3.2 Top-down view of the assembled 5-inch iFlight XL5 quadrotor

Fig. 3.2 and 3.3 show the completed integration. The vertical stack places the companion computer at the top for thermal access and connector reach, the battery in the centre of mass, and the flight controller at the bottom near the frame's geometric centre to minimise the lever arm between the IMU and the centre of gravity. The downward-facing camera protrudes through the underside of the frame on its printed bracket.



Fig. 3.3 Side profile of the airframe showing vertical hardware stack

3.4 Flight Controller and Companion Computer

The flight controller is a Matek H743-SLIM V3, built around an STM32H743 microcontroller and running the ArduCopter firmware of the ArduPilot stack. H743 was chosen because of its computational capacity, dual IMU setup for redundancy, and UARTs that are sufficient in number, including one for the companion computer interface. ArduPilot was preferred to other open source autopilot software due to its extensive documentation on its precision landing feature set, built-in support for the LANDING_TARGET MAVLink message, and comprehensive information available online regarding companion computer integration.

Companion computer: Raspberry Pi 4B 8 GB. The Pi 4B offers four cores of ARM Cortex-A72 @ 1.5 GHz, which is more than enough to run the AprilTag detection, the OpenCV pose and undistortion, fusion, and the MAVLink interface at camera frame rate speed while having room for logging and recording as well. The 8 GB memory version was chosen in order to easily fit the required OpenCV libraries, the detection modules, frame buffer, and video recording in memory. The Pi computer uses Python

3 along with picamera2, pupil-apriltags, OpenCV and pymavlink for image capturing, fiducial detection, pose estimation, undistortion, and the MAVLink interface.

3.5 Camera and Sensor Subsystem

The vision sensor is a Raspberry Pi Camera Module V2, based on the Sony IMX219 sensor with a 3.04 mm fixed focal length. The Pi Camera V2 was selected for its native CSI interface to the Raspberry Pi — avoiding the latency and overhead of USB capture — its low cost, light weight, and adequate resolution. The main disadvantage, which is a rolling shutter that introduces movement artifacts in case of vigorous maneuvers, is stated in Chapter 4 and Chapter 7.

The camera is fixed on the bottom surface of the airframe pointing down directly, whereas the ribbon from the camera is directed at the rear part of the drone. This particular orientation is essential for the transformation matrix, which was built in Chapter 4, since the relative position of the camera axes to the axes of the body was defined by the camera orientation in such a way that the ribbon from the camera would point at the rear end of the drone.

A GEPRC M10-DI GNSS receiver is attached to the aircraft on a raised mast to ensure better visibility and minimize interference from the carbon airframe and the high current cabling of the power system. In addition, the airframe includes a MatekSys 3901-L0X optical flow/rangefinder sensor. Its setup and use is listed as a future improvement as opposed to part of the validated pipeline presented here.

3.6 Communication and Power Architecture

The companion computer communicates with the flight controller through a dedicated UART connection set up on the SERIAL6 port of the H743 operating at 921600 baud. This particular baud rate was chosen to ensure zero serialization lag during delivery of the LANDING_TARGET message, which must be delivered at the same rate as the camera frames. This is because any lag during the delivery of this

information will negatively affect the closed loop feedback process, as the companion computer operates as a MAVLink2 source for the VIO component.

Long-range command-and-control is provided by a 915 MHz ExpressLRS system: a RadioMaster transmitter with a 915 MHz ELRS module communicating with an onboard ExpressLRS receiver. Apart from routine flight piloting, the significance of this connection is in the role of a manual override for safety reasons; at any stage of precision landing, the pilot has full authority to take manual control, and this is the main source of safety during powered flight. The power connections are routed from the 4S battery, where the motors are connected directly through the four-in-one electronic speed controller, while the 5 V electronics like the Raspberry Pi and accessories are connected via a Hobbywing UBEC to lower the battery voltage to 5 V regulated voltage supply. This ensures that the companion computer's power supply is separate from that of the motors to avoid any current fluctuation on the computer.

3.7 Marker Design

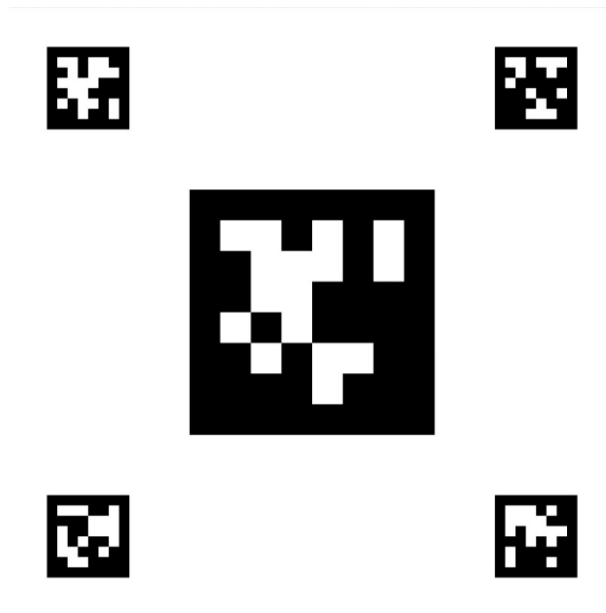


Fig. 3.4 Printed multi-scale AprilTag landing board (5 tags, 60 cm board)

The target for the landing will be an AprilTag design comprising multiple scales, specifically from the tag36h11 series, printed on a board measuring 60 cm on each side. There is a larger 24 cm sized tag (ID 0) located centrally while four smaller 8 cm tags (IDs 1 to 4) are located at the four corners of the square, ± 22 cm away from the centre along each axis. Such an arrangement will solve the range-versus-field-of-view dilemma discussed in Section 1.2, where when the plane is far enough, the central larger tag provides sufficient coverage in pixel terms to allow it to be decoded but the smaller corner tags are out of decoder margin range; it is the dominant tag when approaching for landing.

As the plane descends, the tag grows until it starts to move out of the field of view near touchdown; at this point, the corner tags have grown sufficiently large and can provide landing guidance. Across the intermediate altitude band, several tags are simultaneously visible, providing the redundancy that the fusion estimator exploits. The known fixed geometry of the layout is what makes fusion possible: because the offset of each tag from the board centre is known exactly, every visible tag yields an independent estimate of the same landing-centre point, and these estimates can be combined and cross-checked.

The tag36h11 family was selected over the alternative AprilTag families and over ArUco for the reasons established in the literature review: it offers a large number of distinct, robust codes with high inter-code Hamming distance, strong decoder margins, and resistance to the motion blur and partial occlusion characteristic of aerial imagery. The board layout is summarised in Table 3.2.

Table 3.2 Multi-scale AprilTag board layout (marker-plane frame)

Tag ID (position)	Size (cm)	Offset (x, y) m
0 (centre)	24	(0.00, 0.00)
1 (top-left)	8	(−0.22, +0.22)
2 (top-right)	8	(+0.22, +0.22)
3 (bottom-left)	8	(−0.22, −0.22)
4 (bottom-right)	8	(+0.22, −0.22)

Note: a high-resolution image of the printed multi-scale AprilTag board should be inserted here from the project figure set, with the caption fig. 3.4 The printed multi-scale AprilTag landing board, consisting of one 24 cm central tag (ID 0) and four 8 cm corner tags (IDs 1–4) on a 60 cm board.

3.8 Summary

In summary, this chapter has described a fully integrated 5-inch quadrotor platform in which a Raspberry Pi 4B companion computer performs all perception and a Matek H743-SLIM V3 flight controller retains stabilisation and safety, linked over MAVLink. The multi-scale AprilTag board was shown to be the key design element enabling continuous detection across the descent range

CHAPTER 4

SOFTWARE ARCHITECTURE AND METHODOLOGY

4.1 Introduction

This chapter describes the perception software in detail. It traces the evolution from the Phase 1 ArUco baseline to the Phase 2 multi-tag AprilTag pipeline, and then explains each pipeline stage in turn: camera calibration and undistortion, per-tag pose estimation, the camera-to-body coordinate transformation, the weighted multi-tag fusion estimator, velocity estimation and lag compensation, MAVLink integration, the vibration jump filter, and the deployment, logging, and recording infrastructure.

4.2 Phase 1: ArUco Baseline Pipeline

The first stage of the implementation process involved a traditional recipe of fiducial-landing, which is described extensively across the ArduPilot project. The landing marker consisted of one ArUco marker measuring 30 cm with a size (4×4) and identifier 0. The marker detection was performed using the picamera2 library, followed by converting images to grayscale and marker detection using OpenCV's aruco library with subpixel accuracy. Pose estimation was performed using a built-in function of OpenCV for obtaining a translation vector in the camera coordinate system. Finally, this vector was converted to the body frame of the drone and further represented in angular coordinates as a `LANDING_TARGET` message sent to the flight controller.

The ArUco baseline test showed the effectiveness of testing the whole process from the cam capture to MAVLink receiving of the target location information at the autopilot. As seen in Mission Planner, the flight controller has got a continuous flow of the landing target location data with "PrecLand: Target Found" / "Target Lost" notifications while the marker was in/out of the range. The tracking of the target was stable up to around 7–8 m high with the frame rate of about 15–18 fps.

However, the baseline design had also identified the two basic issues driving the need for the Phase 2 redesign. Firstly, the fact that there was only one fixed marker, at

30 cm, meant that the trade-off between range and field of view could not be better exemplified than in this single component – being easily visible from high altitudes, but also overflowing the field of view at the end stage of approach, thus creating the possibility of interruption of detection right where the most accurate alignment was needed. Secondly, the camera was operated using only approximate parameters instead of properly calibrated ones.

4.3 Phase 2: Multi-Tag AprilTag Pipeline

The pipeline for Phase 2 replaces the use of the ArUco marker with a multi-scale AprilTag structure as described in Section 3.6. It redesigns the perception pipeline based on accuracy and robustness. The complete per-frame pipeline is summarised in fig. 4.1.

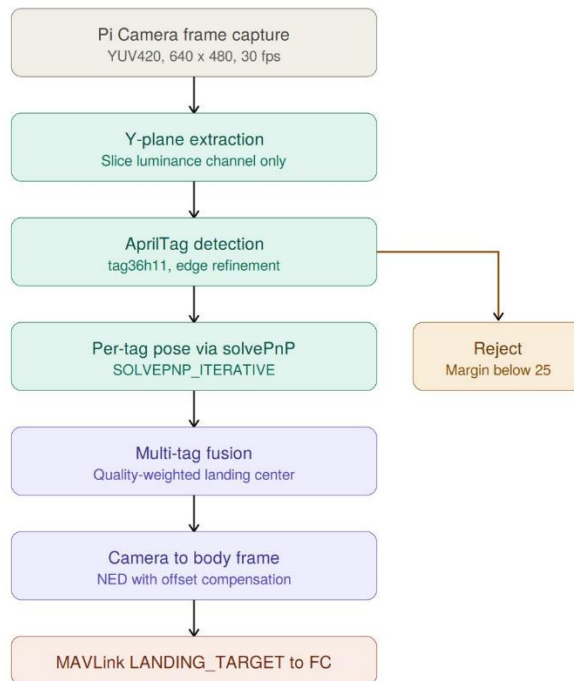


Fig. 4.1 Per-frame software pipeline block diagram

Each captured frame proceeds through the following stages, detailed in the sections that follow: frame capture from the locked sensor mode, luminance-plane

extraction and undistortion, AprilTag detection with decision-margin rejection, per-tag pose estimation via solvePnP, multi-tag fusion into a single landing-centre estimate, transformation to the body frame, jump filtering, and MAVLink dispatch.

4.4 Camera Calibration and Undistortion

Accurate pose estimation from a planar marker depends entirely on an accurate camera model — the focal lengths, principal point, and lens-distortion coefficients that map between the 3D world and 2D image. The Phase 1 baseline’s use of placeholder intrinsics ($f_x = f_y = 530$) was identified as a direct cause of metric error, and Phase 2 therefore begins with a proper calibration.

The camera intrinsics were obtained through a standard chessboard calibration using OpenCV’s `calibrateCamera`. The calibration yielded focal lengths of $f_x = 497.88$ px and $f_y = 496.68$ px, a principal point near the image centre, and radial distortion coefficients $k_1 = 0.140$ and $k_2 = -0.266$, with a root-mean-square reprojection error of 0.255 px. The sub-pixel RMS error indicates a high-quality calibration; a notable practical lesson recorded during this stage was that imperfect printed calibration targets — with small gaps between printed squares — produce internally consistent but metrically meaningless corner detections, so calibration was performed against a pixel-accurate target.

Each captured frame is undistorted before detection using a precomputed remap lookup table generated by OpenCV’s `initUndistortRectifyMap` and applied with `remap`. Precomputing the lookup once and reusing it per frame keeps the per-frame cost to approximately 0.5 ms on the Pi 4. Because the frame is geometrically corrected by this step, the subsequent pose-estimation call is given a null distortion vector — a critical detail, since passing the distortion coefficients again after the image has already been undistorted would apply the correction twice and reintroduce the very error the undistortion removed.

A closely related requirement is sensor-mode locking. The calibrated intrinsics are valid only for the specific sensor read-out mode under which the calibration images

were captured. In other words, if left on its own, the picamera2 library can automatically choose to use a cropped sensor mode that has an effective focal length that doesn't match the calibrated one, thus rendering the intrinsics parameters invalid. In order to prevent this from happening, we manually set the camera into binned full field of view mode (sensor readout size 1640x1232 downsampled to 640x480).

4.5 Per-Tag Pose Estimation

In each frame, the AprilTag detector, configured to find tags from the tag36h11 family using edge refinement, outputs all the detected tags along with their decoded IDs, pixel coordinates of the corners, and a decision margin between 0 and 100 for decoding confidence. In an initial filtering step, detections where the ID is not part of the expected configuration or with decision margin less than 25 are filtered out.

For each filtered detection, pixel coordinates of the four corners are fed into the OpenCV function solvePnP, which calculates for the tag the rotation vector and translation vector relative to the camera, using the size of the tag as input, 24 cm for the center tag and 8 cm for corner tags. Since the actual size of each tag is provided separately, the computed translation vector will be accurate for all tags of different sizes.

The first methodological observation regards the selection of the appropriate PnP solver. The OpenCV library suggests the usage of SOLVEPNP_IPPE_SQUARE for square planar markers. Nevertheless, when using the IMX219 camera sensor, the result of the IPPE_SQUARE solver was erroneous and did not make sense, leading to extremely high reprojection error values. Replacing all calls to the solvePnP function with SOLVEPNP_ITERATIVE solved this issue. It became clear later that an unreasonably large reprojection error indicates a bad choice of solver rather than improper calibration procedure.

4.6 Coordinate-Frame Transformation

The pose returned by solvePnP is expressed in the OpenCV camera frame, in which +x is to the right in the image, +y is downward in the image, and +z points forward out of the lens (and therefore corresponds to altitude above the marker for a downward-facing camera). This must be transformed into the drone's body NED (forward-right-down) frame before it is meaningful to the autopilot.

The transformation follows directly from the physical mounting. With the camera on the underside pointing down and the CSI ribbon toward the tail, the top of the image points toward the drone's nose. A marker located forward of the drone therefore appears toward the top of the image, which corresponds to a negative y-coordinate in the camera frame; a marker to the drone's right appears to the right in the image, a positive x-coordinate. The mapping is given by Equations (4.1)–(4.3):

$$x_{body} = -y_{cam} - x_{offset} \quad (4.1)$$

$$y_{body} = +x_{cam} - y_{offset} \quad (4.2)$$

$$altitude = z_{cam} \quad (4.3)$$

where (x_{offset} , y_{offset} , z_{offset}) is the camera lens position relative to the drone's centre of gravity. The same offset is also supplied to ArduPilot through the PLND_CAM_POS_X/Y/Z parameters, so that the autopilot's internal correction is referenced to the same point as the perception pipeline.

A subtle but consequential issue arose here from a difference in convention between libraries: pupil-apriltags reports the marker frame with its y-axis pointing up, in contrast to OpenCV's y-down convention. Mixing the two silently produces sign-flipped offset errors that remain numerically plausible — the magnitude looks right while the direction is wrong — which is a particularly dangerous class of bug in a control loop. The convention was verified explicitly through bench translation tests, physically moving the marker in known directions and confirming the reported offsets matched.

4.7 Multi-Tag Fusion Estimator

Because the geometry of the marker board is known exactly, each detected tag yields an independent estimate of the same landing-centre point: the known intra-board offset of each tag is subtracted from that tag's camera-frame translation to give the position of the board centre as seen via that tag. The fusion estimator combines these per-tag landing-centre estimates into a single output by a quality-weighted mean.

The weight assigned to each tag's estimate is the product of three factors: the square root of the detected pixel area, the decision margin (normalised to 0–1), and an edge-safety factor. The square root of the pixel area is used rather than the area itself, so that a very large detection does not completely dominate smaller ones — pose accuracy scales sublinearly with marker size in pixels, so a tag four times larger in area is not four times more trustworthy. The decision-margin factor weights more confident decodes more heavily. The edge-safety factor is 1.0 when all four of a tag's corners lie at least 15 px from any image edge, and 0.3 otherwise: detections touching the image boundary yield unstable poses, but during low-altitude approach they may be the only signal available, so they are down-weighted rather than discarded.

Fused landing-centre estimation is the weighted average of the per-tag landing-centre estimates that are valid. The key aspect of the estimator is that it reveals the inconsistency metric, which is the maximum difference between any pair of per-tag landing-centre estimates. The inconsistency metric acts as an instant, real-time measure of internal geometric consistency. If the per-tag estimates agree by a centimetre or two, the fused estimation is reliable; otherwise, something is wrong — either a misdetection, partial occlusion of the tag, or pose ambiguity — before it can contaminate the landing. The inconsistency metric is key to our results in Chapter 6, including the distinction between true surface tilt and pose-induced distortion.

4.8 Velocity Estimation and Lag Compensation

To support the eventual extension to moving platforms, and to smooth the position estimate against measurement noise, the pipeline includes a velocity estimator. It maintains a short history of recent landing-centre positions and fits a quadratic motion model — position, velocity, and acceleration — using weighted least-squares, with exponential weighting that favours the most recent measurements. Outlier measurements are rejected by refitting after discarding residuals beyond three standard deviations, and physically implausible velocity estimates above a speed threshold are suppressed. The estimator also reports a fit-quality value derived from the weighted coefficient of determination.

When the estimated target speed exceeds a minimum threshold and the fit quality is sufficient, the pipeline applies lag compensation: it predicts where the target will be after the system’s characteristic latency (camera, MAVLink, and control response, totalling approximately 0.25 s) using the velocity and acceleration estimates, and sends the predicted position rather than the measured one. For the static and inclined targets validated in this report, the platform is effectively stationary, so prediction is dormant; the machinery is present and exercised as the foundation for the dynamic-platform work identified as future development.

4.9 MAVLink Integration

The fused position estimate is transmitted to the flight controller as a MAVLink `LANDING_TARGET` message in the `MAV_FRAME_BODY_NED` frame. The message carries the angular offsets — the arctangents of the forward and right body offsets divided by the altitude — the straight-line distance to the marker, and the angular size of the largest visible tag, which the autopilot’s rejection logic uses to sanity-check the measurement.

A non-obvious integration issue dominated the early MAVLink work: ArduPilot expects the message timestamp in boot-relative microseconds, not Unix-epoch microseconds. Supplying epoch timestamps caused every message to be silently

rejected by the autopilot, with no error surfaced to the companion computer — an issue diagnosable only by reading the ArduPilot source to discover the expectation. Once the timestamp was reformatted as microseconds since the script's boot reference, messages were accepted and the precision-landing subsystem began to act on them. The message also required non-zero angular-size fields for the same rejection logic to accept it.

4.10 Vibration Jump Filter and Robustness

Powered flight introduces a class of noise entirely absent from bench testing. Airframe vibration, coupled into the rolling-shutter sensor, causes occasional frame-to-frame corner-detection errors that manifest as sudden apparent position jumps of several tens of centimetres. Forwarded uncorrected to the autopilot, these spikes are interpreted as genuine target motion: ArduPilot chases each spike, and the resulting correction excites further vibration, amplifying into divergent oscillation.

To suppress this, a frame-to-frame jump filter rejects any new measurement whose horizontal position differs from the previous accepted measurement by more than 30 cm, or whose altitude differs by more than 40 cm — unless the previous accepted measurement is older than 0.5 s, in which case the new measurement is accepted to allow recovery from a genuine target loss. This simple filter distinguishes physically implausible single-frame jumps (vibration artifacts) from sustained real motion, and was essential to stable behaviour under power. The rotation-matrix-based tilt extraction described in Chapter 6 is similarly designed for robustness: it derives tilt from the marker's Z-axis in a manner invariant to the marker's in-plane yaw, and disambiguates the sign of the Z-axis so that pose-ambiguity sign flips do not corrupt the tilt reading.

4.11 Deployment, Logging, and Recording

The detection process is run as a system service which is automatically triggered during boot and restarted in case of unhandled errors with a restart delay of 10 s and a limit of five restarts per 200 s. Such an approach helps to make the detection process reliable in case of temporary malfunctions.

The pipeline maintains comprehensive logging: a rolling status thread records detection state, fused position, velocity, tilt, and warning counts at regular intervals, and periodic annotated snapshots are written to disk. Offline testing and analysis is facilitated by capturing overlay-burned video, which includes live data such as the position, altitude, tilt, number of tags, and discrepancy value, starting at the time of target acquisition and ending just several seconds after target loss. This is done simultaneously with the flight controller recording the DataFlash logs (.BIN files) on every flight, which are then compared offline to those recorded on the companion computer.

4.12 Summary

This chapter provides an explanation of the software design and approach employed for the UAV precision landing solution. In particular, it explains the development process from the first-phase ArUco landing method to the second-phase multi-tag AprilTag pipeline approach. This includes camera calibration, image undistortion, AprilTag detection, pose estimation, coordinate transform, and MAVLink communication with the flight controller.

A weighted multi-tag fusion approach was devised to increase landing accuracy and robustness by fusing multiple tags to get a landing centre estimate. Some added functions, including velocity estimate, lag correction, vibration jump filtering, and automatic logging, were incorporated to enhance robustness in real-flight conditions. The complete detection pipeline was deployed on the Raspberry Pi companion computer for real-time onboard precision landing.

CHAPTER 5

EXPERIMENTAL SETUP AND TESTING METHODOLOGY

5.1 Introduction

This chapter sets out how the system was tested. It describes the bench-testing procedure used to validate the software stack and verify coordinate conventions, the powered-hover testing methodology using LOITER and LAND flight modes, the ArduPilot precision-landing parameter configuration, and the dual-stream data collection and log-correlation approach used to analyse system behaviour.

5.2 Bench Testing

Before any powered flight, the complete software stack was exercised through bench testing. The camera-and-companion-computer assembly was held above the printed marker board by hand, and the marker was translated, rotated, raised, and lowered through known displacements while the pipeline ran. Bench testing served three purposes. First, it allowed rapid iteration on detection and pose-estimation logic without the cost and risk of flight. Second, it was the setting in which the coordinate-frame conventions were verified: physically moving the marker forward, right, and down and confirming that the reported body-frame offsets changed in the expected directions and signs was the definitive test that caught the y-up/y-down convention mismatch described in Section 4.5. Third, it established the working ranges of detection and the decision-margin behaviour of each tag size before those parameters were trusted in flight.

A custom set of diagnostic scripts supported this stage: a real-time visual detection tester for live inspection of the overlay, and a multi-solver pose-diagnostic script that cross-checked pose estimates and offset computations across solvers and tags. These

were maintained alongside the main detection script and kept synchronised, so that a fix verified in diagnostics could be transferred directly to the flight code.

5.3 Powered Hover Testing

Following bench validation, powered tests were conducted with the aircraft airborne. Two flight modes were used deliberately. LOITER mode was used to position the aircraft over the board and to verify the perception pipeline under genuine powered-flight conditions — vibration, real altitude variation, and outdoor lighting — while the autopilot held position using its own sensors and did not act on the vision corrections. This is the safe configuration in which the full perception chain, including the jump filter and the disagreement metric, was characterised in flight. LAND mode was then used to evaluate closed-loop behaviour, because ArduPilot’s precision-landing corrections are active only in LAND mode; in LOITER the LANDING_TARGET messages are received and logged but not acted upon.

This LOITER-then-LAND methodology is a deliberate risk-management strategy: it allowed the perception pipeline to be fully validated under power before the autopilot was permitted to act on its output, and the 915 MHz ELRS manual-override link was available throughout as the ultimate safety guarantee. Tests were conducted across an altitude range from approximately 1.7 m to 8.6 m above the board, spanning the full operational envelope from terminal approach to maximum detection range, and over both level and inclined board configurations.

5.4 ArduPilot Parameter Configuration

The precision-landing subsystem was configured through the ArduPilot PLND parameter family. Precision landing was enabled (`PLND_ENABLED = 1`) with the companion-computer source selected (`PLND_TYPE = 1`) and the estimator and strictness parameters set (`PLND_EST_TYPE = 1`, `PLND_STRICT = 1`). The system lag parameter was set to match the measured pipeline latency (`PLND_LAG = 0.25 s`),

and the camera mounting offset was supplied through PLND_CAM_POS_X/Y/Z so that the autopilot's internal correction referenced the same point as the perception pipeline. The descent speed was set conservatively (LAND_SPEED = 30 cm/s, WPNAV_SPEED_DN = 30) to give the correction loop time to act during descent. The key parameters are summarised in Table 5.1.

Table 5.1 ArduPilot precision-landing parameter configuration

Parameter	Value	Function
PLND_ENABLED	1	Precision landing enabled
PLND_TYPE	1	Companion-computer source
PLND_EST_TYPE	1	Estimator type
PLND_STRICT	1	Strict measurement acceptance
PLND_LAG	0.25 s	System latency compensation
PLND_CAM_POS_Z	-0.05 m	Camera offset below CG
LAND_SPEED	30 cm/s	Final descent speed
WPNAV_SPEED_DN	30 cm/s	Waypoint descent speed

The measurement-noise parameter PLND_ACC_P_NSE governs how aggressively the autopilot trusts and acts upon the vision corrections. This parameter is identified in the analysis of Chapter 6 as a key tuning lever: an aggressive setting, combined with the residual measurement noise of powered flight, contributes to underdamped position response, and reducing it is the principal recommendation for stabilising future closed-loop autonomous descent.

A standing safety item recorded throughout development concerns the arming checks. During bench and tethered development the arming checks were relaxed for convenience; re-enabling the full ArduPilot arming-check suite is a mandatory

prerequisite before any unsupervised outdoor autonomous flight, and is noted here and in the future-work chapter as a non-negotiable safety gate.

5.5 Data Collection and Log Correlation

Each flight produced two parallel data streams. The companion computer logged its perception output — detection state, per-frame fused position, altitude, velocity, tilt, tag count, and the disagreement metric — together with periodic annotated snapshots and overlay-burned video. In addition, the flight controller recorded its own logs in DataFlash format (BIN), which included the state of the platform itself and the received LANDING_TARGET stream together with the autopilot response. It was possible to correlate this data in post-flight analysis and assign the observed behavior to the right subsystem: e.g., determine whether a particular oscillation occurred because of noisy input to the autopilot, not in the perception chain, or a particular detection failure occurred at a certain altitude and attitude.

5.6 Summary

To conclude, this chapter has described an effective test process, which is a systematic and progressive approach for validating the entire precision-landing capability in a safe and controlled environment. The testing was done through various stages, starting from bench-level validation of the perception capability and communication capability, to actual powered-flight tests. The ground tests verified the ability of the perception capability to accurately detect AprilTag targets, estimate their position correctly, transmit data via MAVLink, align the coordinates, and generally operate in a stable way. This systematic approach helped reduce the integration risk and ensure reliability of key components, before going airborne.

Once the system was validated on the ground, it was time to perform tests during actual flights using the LOITER-then-LAND sequence of operations. In this test procedure, the UAV was flown in a stationary hover mode (LOITER mode), where the perception

module could detect and confirm the location of the landing site, after which the UAV transitioned to the autonomous landing phase (LAND mode).

This test procedure was additionally bolstered through a detailed setup of ArduPilot precise landing parameters and two-channel logging infrastructure. Logs obtained from the on-board perception computer alongside the flight controller were captured simultaneously, allowing for the correlation of visual object detection, landing target estimate transmission, and flight controller actions during post-test analysis. Logging in this manner was critical to the debugging process as many synchronization problems, pose estimation problems, and integrations were diagnosed through this approach.

In sum, this method provided the capability to examine the perception algorithm in actual flight scenarios while retaining manual override control by the pilot as the core safety feature during tests.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction

This chapter presents the results of bench and powered-hover testing, organised from hardware verification through baseline detection, multi-tag fusion performance, altitude continuity, inclined-platform tilt estimation, and the documented integration pitfalls, closing with an overall discussion.

6.2 Hardware Implementation Results

The UAV hardware was successfully built and integrated using the 5-inch iFlight XL5 airframe, the Matek H743-SLIM V3 autopilot, the Raspberry Pi 4B copter, the Pi Camera V2, the GEPRC GNSS, and the ExpressLRS radio, with the latter four components non-structurally mounted via customized 3D prints to increase rigidity and reduce vibration. The final UAV can be seen in Figures 3.2 and 3.3. The vertical stack ensured a low and centralized center of gravity, while the Pi Camera V2 was rigidly attached in a specific, known configuration to ensure that the required coordinate-transformation sequence could be performed. The connection between the companion computer and the autopilot via the MAVLink protocol was successfully achieved and reliable throughout the tests, with the autopilot successfully receiving the LANDING_TARGET stream and responding with found/lost notifications.

6.3 ArUco Baseline Detection Results

The Phase 1 ArUco baseline validated the complete hardware-and-software chain. The Pi Camera detected the single 30 cm ArUco marker in real time, recovered its pose, and transmitted position offsets to the flight controller, which Mission Planner confirmed receiving as a stable landing-target stream. Detection remained stable to approximately 7–8 m altitude under outdoor lighting, with onboard processing sustaining roughly 15–18 frames per second.

The baseline equally confirmed the two limitations that motivated Phase 2. The single fixed marker size could not simultaneously satisfy the altitude and terminal-descent detection requirements, and the uncalibrated pipeline produced metrically incorrect distances. The latter is quantified directly in Section 6.6. Together these results justified the migration to the multi-scale AprilTag system.

6.4 AprilTag Multi-Tag Detection and Fusion Results

The Phase 2 multi-tag pipeline was evaluated through powered hover flights across the full altitude range. The primary internal-consistency metric is the disagreement between landing-centre estimates derived from different visible tags — the maximum pairwise distance among per-tag estimates. Where multiple tags were simultaneously detected, this metric remained tight, confirming the geometric self-consistency of the fusion framework on real hardware. Table 6.1 summarises representative fusion measurements across the tested altitude range.

Table 6.1 Representative multi-tag fusion measurements (level board)

Altitude (m)	Tags visible	Disagreement (cm)	Offset X,Y (m)
1.74	5	5.0	(−0.30, +0.01)
2.46	5	1.6	(−0.54, −0.18)
3.63	2	1.1	(−0.17, −0.01)
8.56	1	—	(−0.80, −0.62)



Fig. 6.1 Low-altitude detection at 1.74 m with all five tags visible



Fig. 6.2 Mid-altitude two-tag fusion at 3.63 m

As shown in fig. 6.1 and 6.2, the disagreement metric measured 5.0 cm at 1.74 m altitude with all five tags visible, and 1.1 cm at 3.63 m altitude with two tags visible. It is important to interpret this metric correctly: it represents geometric self-consistency among the tags, not absolute accuracy against an external ground truth. Errors that

affect all tags identically — most notably focal-length miscalibration — do not appear in the disagreement metric, even though they would affect absolute landing accuracy. The metric's value is as a real-time confidence indicator and as evidence that the fusion geometry is internally correct; absolute metric accuracy is established separately by the calibration of Section 6.6, after which reported positions matched tape-measure ground truth within a few centimetres across the tested range.

6.5 Detection Continuity Across Altitude

Detection continuity — the central design goal of the multi-scale layout — was confirmed through repeated flights. The central 24 cm tag was reliably detected from over 8.5 m altitude down to approximately 0.6 m, below which it begins to leave the field of view. The 8 cm corner tags were detected reliably from approximately 3.5 m down to terminal descent. The two ranges overlap substantially through the mid-altitude band, so that at no point in a nominal descent is the system without a usable detection, and the transition between high-altitude single-tag operation and low-altitude corner-tag operation is smooth rather than abrupt.



Fig. 6.3 Maximum-altitude single-tag detection at 8.56 m

fig 6.3 shows the maximum detection altitude observed, 8.56 m, at which only the central tag is decoded — comfortably beyond the operational range required for a typical landing approach.

6.6 Single-Tag Tilt Instability and Inclined-Platform Detection

A central result of this work concerns the recovery of marker-surface tilt, and the clear distinction the disagreement metric draws between two superficially similar phenomena: a spurious tilt artifact arising from single-tag pose geometry, and genuine tilt arising from a physically inclined landing surface.

6.6.1 The single-tag tilt artifact

When only the central tag is detected and its pixel area is small relative to the image — that is, at higher altitude — the solvePnP pose becomes geometrically ill-conditioned, and the recovered tilt angles can be substantially larger than the airframe's true attitude.



Fig. 6.4 Single-tag tilt instability at 6.28 m over a level board

Fig. 6.4 illustrates this regime: a reported pitch of -21.4° on a physically level marker at 6.28 m altitude. This is consistent with the well-known behaviour of planar PnP in the small-marker limit, where small corner-localisation errors translate into large apparent rotations. Critically, in this regime only a single tag is visible, so the disagreement metric is unavailable — there is no second estimate to cross-check against. This is precisely the failure mode the multi-scale layout is designed to mitigate at descent altitudes, where the corner tags become detectable and supply the cross-check.

6.6.2 Genuine inclined-surface tilt under full fusion

To validate that the pipeline recovers real surface inclination — as distinct from the artifact above — the board was physically inclined by propping one edge on a raised object, and the aircraft was flown over it. The discriminating evidence is the behaviour of tilt together with the disagreement metric under full multi-tag fusion.



Fig. 6.5 Inclined-platform detection at 1.68 m with full five-tag fusion

The contrast with Section 6.5.1 is decisive. For the artifact case, the large tilt appears only when a single small tag is visible and there is no disagreement metric to contradict it; on a level board, frames with full five-tag fusion report tilt near zero. For the inclined board Fig. 6.5, the large tilt persists at -15.8° even with all five tags fused and a per-tag disagreement of just 0.9 cm. If this tilt were a pose artifact, fusion across five spatially separated tags would average it toward zero; instead it remains, and the five tags agree to sub-centimetre precision about an inclined plane. The disagreement metric thus serves as the discriminator: low disagreement with persistent tilt is genuine surface inclination; absent disagreement (single tag) with large tilt is the artifact.

This behaviour was confirmed across a descent sequence over the inclined board, with the reported pitch persisting in the -13° to -21° band while the disagreement metric remained low (0.8–2.0 cm) in every multi-tag frame from 0.84 m to 3.36 m altitude, as summarised in Table 6.2. The measured tilt is therefore a consistent property of the inclined surface and not a function of how many tags happen to be visible.

Table 6.2 Inclined-platform descent sequence: tilt persists under fusion

Altitude (m)	Tags visible	Disagreement (cm)	Pitch ($^\circ$)
3.36	1	—	-16.9
2.46	5	1.6	-13.7
1.68	5	0.9	-15.8
0.84	5	0.8	-16.0
2.29	5	2.0	-21.8

This result demonstrates that the rotation-matrix-based, yaw-invariant tilt extraction correctly recovers the surface normal of a non-level landing target. The capability is directly relevant to landing on sloped terrain, ramps, vehicle surfaces, and uneven recovery sites — scenarios the level-board precision-landing literature largely

does not address. The present work validates detection and surface-tilt estimation over the incline; closed-loop autonomous touchdown onto a sloped surface is identified as future work.

6.7 Documented Integration Pitfalls

A distinguishing contribution of this work is the explicit documentation of hardware-integration pitfalls that are common in deployment but underdocumented in the literature. Each was diagnosed and resolved during testing, and is recorded here as a practical reference.

Camera miscalibration and altitude-dependent scale error. The Phase 1 pipeline used placeholder intrinsics ($f_x = f_y = 530$) instead of calibrated values for the IMX219. This produced an altitude-dependent scale error: reported distances matched ground truth within a few centimetres below 1.5 m, but at approximately 2.8 m the reported altitude was roughly $2.7\times$ smaller than the true altitude. Receiving an underestimated distance, the autopilot applied overly aggressive horizontal corrections that destabilised the airframe. Replacing the placeholders with chessboard-calibrated intrinsics eliminated the scale error, after which positions matched tape-measure ground truth within a few centimetres across the tested range.

PnP Solvers Mismatch. The suggested choice for planar square markers was `SOLVEPNP_IPPE_SQUARE`, but it resulted in inconsistent reprojection errors using the IMX219 sensor; this was addressed with `SOLVEPNP_ITERATIVE`, which will be used henceforth. Large reprojection errors have been found to indicate solver mismatch, rather than calibration problems.

Sign conventions in coordinate frame representations. The pupil-apriltags marker frame uses y pointing upwards, unlike the y -down representation used by OpenCV. Combining both approaches caused silent offsets in sign of the displacement due to a bug in the control loop where displacements were visually plausible (correct magnitude but wrong direction). Tests involving physical translations on the bench allowed diagnosing this problem.

Pose ambiguity of square markers. The solution of solvePnP for a square marker suffers from a pose ambiguity: the output pose could be such that the marker's z-axis points away from the camera, meaning that the tilt value jumps back and forth between high positive and negative values when tracking the same motionless marker from frame to frame. Properly normalizing the rotation to ensure the marker z-axis points towards the camera solves this problem.

MAVLink timestamp formatting. ArduPilot expects boot-relative microsecond timestamps in the LANDING_TARGET message; supplying Unix-epoch microseconds caused every message to be silently rejected, with no error surfaced to the companion computer — diagnosable only by reading the autopilot source.

Sensor-mode locking. Left to default, picamera2 may select a cropped sensor mode whose effective focal length differs from the calibrated value, silently invalidating the intrinsics. Explicitly locking the full-FOV binned mode and verifying the field of view at startup prevents this.

Vibration-induced detection noise. In powered flight, vibration coupled through the rolling-shutter sensor produces occasional position spikes of several tens of centimetres. Without the frame-to-frame jump filter, the autopilot chases these spikes and amplifies them into divergent oscillation; the filter (Section 4.9) suppresses them while preserving genuine motion.

6.8 Discussion

The results collectively demonstrate that a low-cost, fully onboard precision-landing system can achieve geometrically self-consistent, metrically accurate relative-position estimation across the full descent range on resource-constrained hardware. The multi-scale design of the AprilTag addresses the range-field-of-view dilemma, the calibrated and distortion-free camera provides metering precision, the weighted fusion algorithm fuses multiple measurements to form an estimate of good quality and reveals a real-time confidence measure, and the jump filter ensures stability when power is applied.

There are two discoveries of note that go beyond simple refinement. One is that the disagreement statistic is not only useful as a confidence measure but as a way to discriminate between real and false indications of inclined surfaces, i.e., between true incline angles and errors due to tag pose.

Second, the work's emphasis on documented integration pitfalls reflects a broader lesson: vision-based precision landing is, in practice, dominated by hardware-level integration concerns rather than algorithmic novelty. Each of the pitfalls mentioned, if not taken care of, was individually enough to cause inaccuracy or destabilize the system, and none of them is well-represented in theoretical concepts. The present system provides a practical starting point for precise landings on such platforms.

6.9 Summary

In summary, this chapter has shown multi-tag fusion in a geometrically consistent manner (5.0 cm tolerance at 1.74 m, 1.1 cm tolerance at 3.63 m), continuous target tracking from 8.56 m to terminal descent, and a careful separation of single tag tilt artifact from true surface tilt based on the discrepancy criterion. Integration issues illustrated here support the claim that practical precision landing is more concerned with hardware integration than anything else.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

In this work, we propose and implement the architecture, hardware, and evaluation of an integrated precision landing system onboard a small multirotor UAV using the development process of an ArUco benchmarking stage and an AprilTag multi-scale fusion stage. The implementation was done using a 5-inches iFlight XL5 multirotor equipped with Matek H743-SLIM V3 flight controller using ArduPilot, Raspberry Pi 4B computer along with Pi Camera V2 interfaced via MAVLink LANDING_TARGET message protocol.

Phase 1's ArUco baseline successfully characterized the end-to-end hardware-software pipeline chain, and identified the shortcomings – single-marker range limitation and lack of scale calibration – that would become the key challenges for the Phase 2 design. This multi-scale AprilTag implementation, featuring a calibrated camera model, per-frame distortion correction, locked sensor mode, multi-marker pose fusion with weighting, and a vibration jump filter, addressed these challenges. In powered hover tests, multi-marker pose consistency was found to be 5.0 cm at 1.74 m altitude with all five markers visible, and 1.1 cm at 3.63 m with only two markers visible, which constituted robust evidence for geometric consistency. Central marker detection consistently extended to 8.56 m altitude range, significantly above the range required for most other methods. Finally, the paper highlights a number of less obvious integration challenges that were encountered in the course of development, along with the diagnostic approach used to identify and address each of them.

7.2 Future Work

Some obvious directions for future research and development stem from the present work. The first and most pressing task is that of completing and analyzing fully closed-loop autonomous touchdowns, including additional tuning of the ArduPilot parameters

responsible for precise landing, especially decreasing the value of $PLND_ACC_P_NSE$ in order to eliminate excessive corrections in position and attain a smoother terminal descent. In particular, future experiments should include systematic evaluation of the accuracy of landing relative to ground truth at different trials of landing either on a flat or on an inclined surface.

Furthermore, another important task is adding other sensors to improve the descent. Using a small rangefinder, like, e.g., TF-Luna or TFmini-S, would give better estimation of altitude during the last sub-meter descent when vision becomes less accurate and reliable due to the increased amount of errors. In addition, using the MatekSys 3901-L0X optical flow sensor can contribute to improving the performance of position control during precise landing.

One other possible development in the future is taking the system towards performing landings onto moving platforms. This is because the existing perception pipeline takes into consideration the velocity and lag compensation systems, which create a good base for tracking moving landing platforms like autonomous vehicles and docking sites. If the UAV manages to perform landings from such platforms, then its uses will increase greatly.

Another good direction for improvement would be the implementation of a Kalman filter for the inertial sensor data fusion with the vision processing system. This will help to get improved positioning and reduced measurement noise even when the markers are not visible for a short while.

Hardware upgrades also provide avenues to improve robustness. The adoption of a global shutter or high dynamic range imaging camera is expected to help lower motion blur, enable accurate detection in adverse light conditions, and enhance robustness when executing fast maneuvers.

Lastly, the autonomous precision landing system can be embedded into a full autonomous aerial mission consisting of an auto takeoff, auto navigation to a target,

target localization, and autonomous precision landing. This will allow the development of an end-to-end autonomous aircraft that could potentially handle package deliveries, automatic docking, battery charging stations, among other tasks.

REFERENCES

- [1] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [2] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 3400–3407.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and robust fiducial detection,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2016, pp. 4193–4198.
- [4] M. Kalaitzakis, B. Cain, S. Carroll, A. Ambrosi, C. Whitehead, and N. Vitzilaios, “Fiducial markers for pose estimation: Overview, applications and experimental comparison of the ARTag, AprilTag, ArUco and STag markers,” *J. Intelligent & Robotic Systems*, vol. 101, no. 4, pp. 1–26, 2021.
- [5] M. Krogus, A. Haggemiller, and E. Olson, “Flexible layouts for fiducial tags,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2019, pp. 1898–1903.
- [6] C. Patruno, M. Nitti, A. Petitti, E. Stella, and T. D’Orazio, “A vision-based approach for unmanned aerial vehicle landing,” *J. Intelligent & Robotic Systems*, vol. 95, pp. 645–664, 2019.
- [7] A. Marut, K. Wójtowicz, and K. Falkowski, “ArUco markers pose estimation in UAV landing aid system,” in *Proc. IEEE Int. Workshop on Metrology for AeroSpace*, 2019, pp. 261–266.
- [8] J. Springer and M. Kyas, “Autonomous drone landing with fiducial markers and a gimbal-mounted camera for active tracking,” in *Proc. IEEE Int. Conf. on Autonomous Robot Systems and Competitions (ICARSC)*, 2022, pp. 243–249.
- [9] S. Lange, N. Sünderhauf, and P. Protzel, “A vision-based onboard approach for landing and position control of an autonomous multirotor UAV in GPS-denied environments,” in *Proc. Int. Conf. on Advanced Robotics (ICAR)*, 2009, pp. 1–6.
- [10] A. Borowczyk, D. T. Nguyen, A. Phu-Van Nguyen, D. Q. Nguyen, D. Saussié, and J. Le Ny, “Autonomous landing of a quadcopter on a high-speed ground vehicle,” *J. Guidance, Control, and Dynamics*, vol. 40, no. 9, pp. 2378–2385, 2017.
- [11] D. Falanga, A. Zanchettin, A. Simovic, J. Delmerico, and D. Scaramuzza, “Vision-based autonomous quadrotor landing on a moving platform,” in *Proc. IEEE Int. Symp. Safety, Security, and Rescue Robotics (SSRR)*, 2017, pp. 200–207.

- [12] H. Yang, J. Liu, and D. Gu, “Vision-based autonomous navigation for quadrotor UAVs in unknown and constrained environments,” *Int. J. Intelligent Robotics and Applications*, 2025.
- [13] S. Sepahvand, N. Amiri, and F. Janabi-Sharifi, “Deep visual servoing of an aerial robot using keypoint feature extraction,” *arXiv preprint arXiv:2503.23171*, 2025.
- [14] M. Luo, T. Qin, and S. Shen, “Monocular visual-inertial state estimation for autonomous drone landing,” *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 5333–5340, 2020.
- [15] A. Baca, P. Štěpán, and R. Pěnička, “Embedded computer vision for UAV autonomous landing using MAVLink and companion computers,” *J. Field Robotics*, vol. 39, no. 4, pp. 512–526, 2022.
- [16] P. Corke, “Fiducial markers and pose estimation for robotics applications,” *IEEE Robotics & Automation Magazine*, vol. 25, no. 2, pp. 77–89, 2019.
- [17] D. Scaramuzza and F. Fraundorfer, “Visual odometry: Part I — The first 30 years,” *IEEE Robotics & Automation Magazine*, vol. 18, no. 4, pp. 80–92, 2011.
- [18] A. Kendoul, “Survey of advances in guidance, navigation, and control of unmanned rotorcraft systems,” *J. Field Robotics*, vol. 29, no. 2, pp. 315–378, 2012.
- [19] ArduPilot Development Team, “Precision Landing with Companion Computer,” *ArduPilot Documentation*, 2024. [Online]. Available: <https://ardupilot.org>
- [20] MAVLink, “MAVLink messaging documentation and APIs.” [Online]. Available: <https://mavlink.io>
- [21] OpenCV Developers, “OpenCV ArUco and camera-calibration module documentation.” [Online]. Available: <https://docs.opencv.org>
- [22] M. T. R. Khan, M. M. M. Manhas, et al., “Simulation and stabilization of a custom-made quadcopter in Gazebo using ArduPilot and QGroundControl,” *Amrita Vishwa Vidyapeetham Publication*, 2023.
- [23] A. A. N. Kumaar and S. Kochuvila, “Mobile service robot path planning using deep reinforcement learning,” *IEEE Access*, vol. 11, pp. 100083–100096, 2023.
- [24] G. Varghese, T. G. C. Reddy, A. K. Menon, A. Paul, S. Kochuvila, V. D. R., R. Bhat, and N. Kumar, “Multi-robot system for mapping and localization,” in *Proc. 2023 8th Int. Conf. on Robotics and Automation Engineering (ICRAE)*, 2023, pp. 79–84.